



Foundation University Islamabad

School of Science and Technology

Name: Hunaina Yasir

Subject: OOP Lab

Roll Number: 073

Section: B

OPEN ENDED LAB

TASK 1:

Create a class Student that stores information about a student, including their name (string), roll number (int), and marks (float). The class should have a constructor that initializes these attributes and a destructor that displays a message when a student object is destroyed. Implement a function display() that prints the student details. In the main() function, create three student objects, take their details as input, and display them using the display() function. Observe the destructor being called automatically when the program ends.

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string name;
    int rollNumber;
    float marks;

public:
    Student(string n, int r, float m) {
        name = n;
        rollNumber = r;
        marks = m;
    }

    ~Student() {
        cout << "Destructor called: Student object for " << name << " is being destroyed." << endl;
    }

    void display() {
        cout << "Name: " << name << ", Roll No: " << rollNumber << ", Marks: " << marks << endl;
    }
}
```

```

};

int main() {
    string n;
    int r;
    float m;

    for (int i = 1; i <= 3; i++) {
        cout << "Enter details for Student " << i << " (Name, RollNo, Marks): ";
        cin >> n >> r >> m;
        Student s(n, r, m);
        s.display();
    }

    return 0;
}

```

```

2  #include <string>
3  using namespace std;
4
5  class Student {
6  private:
7      string name;
8      int rollNumber;
9      float marks;
10
11 public:
12     Student(string n, int r, float m) {
13         name = n;
14         rollNumber = r;
15         marks = m;
16     }
17
18     ~Student() {
19         cout << "Destructor called: Student object for " << name << " is being destroyed." << endl;
20     }
21
22     void display() {
23         cout << "Name: " << name << ", Roll No: " << rollNumber << ", Marks: " << marks << endl;
24     }
25 };
26
27 int main() {
28     string n;
29     int r;
30     float m;
31
32     for (int i = 1; i <= 3; i++) {
33         cout << "Enter details for Student " << i << " (Name, RollNo, Marks): ";
34         cin >> n >> r >> m;
35         Student s(n, r, m);
36         s.display();
37     }
38
39     return 0;
40 }

```

OUTPUT:

```

Enter details for Student 1 (Name, RollNo, Marks): aiza 76 456
Name: aiza, Roll No: 76, Marks: 456
Destructor called: Student object for aiza is being destroyed.
Enter details for Student 2 (Name, RollNo, Marks): leo 56 66
Name: leo, Roll No: 56, Marks: 66
Destructor called: Student object for leo is being destroyed.
Enter details for Student 3 (Name, RollNo, Marks): iera 34 99
Name: iera, Roll No: 34, Marks: 99
Destructor called: Student object for iera is being destroyed.

```

```
Enter details for Student 1 (Name, RollNo, Marks): aiza 76 456
Name: aiza, Roll No: 76, Marks: 456
Destructor called: Student object for aiza is being destroyed.
Enter details for Student 2 (Name, RollNo, Marks): leo 56 66
Name: leo, Roll No: 56, Marks: 66
Destructor called: Student object for leo is being destroyed.
Enter details for Student 3 (Name, RollNo, Marks): iera 34 99
Name: iera, Roll No: 34, Marks: 99
Destructor called: Student object for iera is being destroyed.
```

TASK 2:

Design a class BankAccount to manage bank accounts with attributes for account number (int), account holder name (string), and balance (float). Implement a parametrized constructor to initialize an account with values taken from the user. Implement member functions deposit(float amount) to add money and withdraw(float amount) to deduct money, ensuring that withdrawals do not exceed the current balance. In the main() function, create a BankAccount object, perform at least one deposit and one withdrawal, and display the final balance. The destructor should confirm that account details have been saved successfully.

SOURCE CODE:

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:
    int accountNum;
    string holderName;
    float balance;

public:
    BankAccount(int acc, string name, float bal) {
        accountNum = acc;
        holderName = name;
        balance = bal;
    }

    ~BankAccount() {
        cout << "Account details for " << holderName << " saved successfully. Destructor called." << endl;
    }

    void deposit(float amount) {
        balance += amount;
        cout << "Deposited: " << amount << ". New Balance: " << balance << endl;
    }
}
```

```

void withdraw(float amount) {
    if (amount <= balance) {
        balance -= amount;
        cout << "Withdrew: " << amount << ". Remaining Balance: " << balance << endl;
    } else {
        cout << "Insufficient balance!" << endl;
    }
}

void showBalance() {
    cout << "Final Balance: " << balance << endl;
}

};

int main() {
    int acc;
    string name;
    float bal;

    cout << "Enter Account Number, Name, and Initial Balance: ";
    cin >> acc >> name >> bal;

    BankAccount myAccount(acc, name, bal);
    myAccount.deposit(500.0);
    myAccount.withdraw(200.0);
    myAccount.showBalance();

    return 0;
}

```

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class BankAccount {
6  private:
7      int accountNum;
8      string holderName;
9      float balance;
10
11 public:
12     BankAccount(int acc, string name, float bal) {
13         accountNum = acc;
14         holderName = name;
15         balance = bal;
16     }
17
18     ~BankAccount() {
19         cout << "Account details for " << holderName
20         << " saved successfully. Destructor called." << endl;
21     }
22
23     void deposit(float amount) {
24         balance += amount;
25         cout << "Deposited: " << amount << ". New Balance: " << balance << endl;
26     }
27
28     void withdraw(float amount) {
29         if (amount <= balance) {
30             balance -= amount;
31             cout << "Withdrew: " << amount
32             << ". Remaining Balance: " << balance << endl;
33         } else {
34             cout << "Insufficient balance!" << endl;
35         }
36     }
37 }

```

```

35     }
36 }
37
38 void showBalance() {
39     cout << "Final Balance: " << balance << endl;
40 }
41 };
42
43 int main() {
44     int acc;
45     string name;
46     float bal;
47
48     cout << "Enter Account Number, Name, and Initial Balance: ";
49     cin >> acc >> name >> bal;
50
51     BankAccount myAccount(acc, name, bal);
52     myAccount.deposit(500.0);
53     myAccount.withdraw(200.0);
54     myAccount.showBalance();
55
56     return 0;
57 }

```

OUTPUT:

Enter Account Number, Name, and Initial Balance: 456 Lia 67000
Deposited: 500. New Balance: 67500
Withdrew: 200. Remaining Balance: 67300
Final Balance: 67300
Account details for Lia saved successfully.

```
Enter Account Number, Name, and Initial Balance: 456 Lia 67000
Deposited: 500. New Balance: 67500
Withdrew: 200. Remaining Balance: 67300
Final Balance: 67300
Account details for Lia saved successfully.
```

TASK 3:

Create a base class Shape with attributes for length and width (both float) and a constructor to initialize them. Then, derive a class Rectangle from Shape, which will use the inherited attributes to calculate the area of the rectangle. The Rectangle class should have a constructor that initializes the values of length and width using the base class constructor. Implement a function calculateArea() in Rectangle to compute and display the area. The destructor should print a message when an object is destroyed. In the main() function, create an object of Rectangle, initialize it with values, compute the area, and observe the destructor being called automatically when the program ends.

SOURCE CODE:

```
#include <iostream>
using namespace std;

class Shape {
protected:
    float length;
    float width;

public:
    Shape(float l, float w) {
        length = l;
        width = w;
    }
};

class Rectangle : public Shape {
public:
    Rectangle(float l, float w) : Shape(l, w) {
    }

    ~Rectangle() {
        cout << "Rectangle object destroyed." << endl;
    }

    void calculateArea() {
        float area = length * width;
```

```

        cout << "Area of Rectangle: " << area << endl;
    }
};

int main() {
    float l, w;
    cout << "Enter length and width: ";
    cin >> l >> w;

    Rectangle r1(l, w);
    r1.calculateArea();

    return 0;
}

```

```

2   using namespace std;
3
4   class Shape {
5   protected:
6       float length;
7       float width;
8
9   public:
10      Shape(float l, float w) {
11          length = l;
12          width = w;
13      }
14  };
15
16  class Rectangle : public Shape {
17  public:
18      Rectangle(float l, float w) : Shape(l, w) {
19      }
20
21      ~Rectangle() {
22          cout << "Rectangle object destroyed." << endl;
23      }
24
25      void calculateArea() {
26          float area = length * width;
27          cout << "Area of Rectangle: " << area << endl;
28      }
29  };
30
31  int main() {
32      float l, w;
33      cout << "Enter length and width: ";
34      cin >> l >> w;
35
36      Rectangle r1(l, w);
37      r1.calculateArea();
38
39      return 0;
40  }

```

OUTPUT:

```

Enter length and width: 45 67
Area of Rectangle: 3015
Rectangle object destroyed.

```

```

Enter length and width: 45 67
Area of Rectangle: 3015
Rectangle object destroyed.

```